Travel Agency Website — ARCHITECTURE.md

Version: 1.0
Status: Production Architecture Specification
Architecture Style: Modular Laravel Monolith + Selective Vue.js
Backend: PHP / Laravel
Database: MySQL
Frontend: Blade + Vue.js
Hosting: Hostinger
Domain: Name.com
Development Environment: Google Antigravity
Repository: Git / GitHub
Primary Goal: Fast, SEO-first, secure and scalable travel agency platform

---

1. Architecture Overview

The application will use a **Laravel-first modular monolith architecture**.

The public-facing website will primarily use Laravel Blade/server-rendered HTML for:

- SEO
- Fast first paint
- Low JavaScript dependency
- Easy content indexing
- Better performance on mobile networks

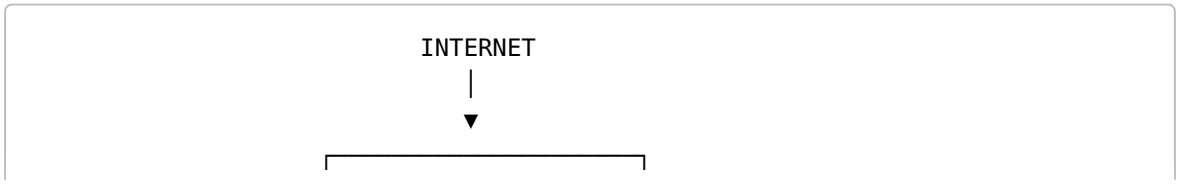
Vue.js will be used selectively for:

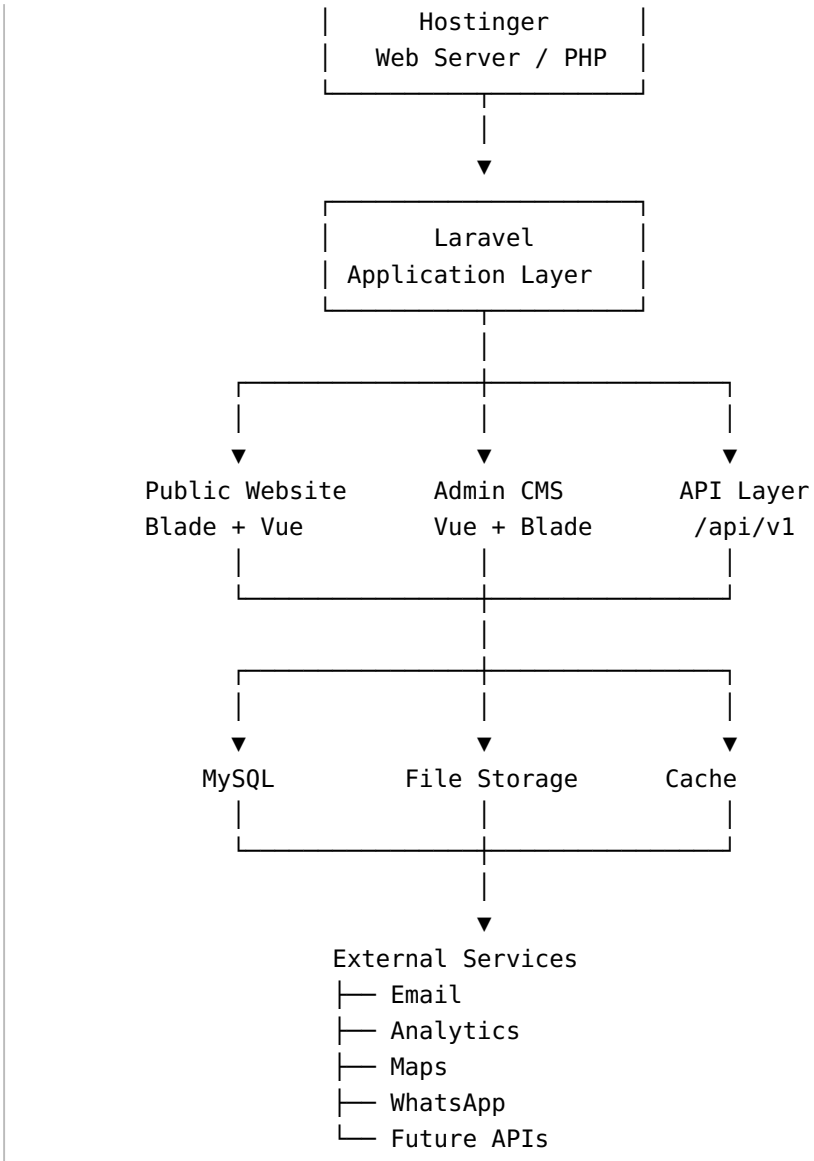
- Interactive filters
- Search
- Dynamic forms
- Visa application wizard
- Document upload
- Gallery interactions
- Admin dashboard
- Real-time-ish UI state changes

The entire public website should NOT be implemented as a heavy SPA.

---

2. High-Level Architecture





---

3. Core Architectural Principle

The application follows:

```
Presentation
     ↓
HTTP / Routing
     ↓
Controllers
     ↓
Form Requests
     ↓
Services
     ↓
```

```
Repositories / Query Layer
     ↓
Models
     ↓
MySQL
```

Business logic should NOT be placed directly inside Blade templates or large controllers.

---

4. Application Domains

The application should be logically divided into domains:

```
01. Website
02. Tours
03. Destinations
04. Visa
05. Air Ticketing
06. Inquiries
07. Content
08. Gallery
09. Testimonials
10. SEO
11. Media
12. Users & Roles
13. Notifications
14. Settings
15. Audit Logs
```

---

5. Recommended Laravel Structure

```
app/
├── Console/
├── Exceptions/
├── Http/
│   ├── Controllers/
│   │   ├── Public/
│   │   ├── Admin/
│   │   └── Api/
│   │
│   ├── Middleware/
│   └── Requests/
│
├── Models/
│
```

```
├── Services/
│   ├── TourService.php
│   ├── VisaService.php
│   ├── InquiryService.php
│   ├── TicketRequestService.php
│   ├── MediaService.php
│   ├── SeoService.php
│   └── NotificationService.php
│
├── Repositories/
│
├── Policies/
│
├── Notifications/
│
├── Jobs/
│
├── Events/
│
├── Listeners/
│
└── Support/
```

---

6. Routes Architecture

Public Routes

```
/
 /about
 /tours
 /tours/{slug}
 /destinations
 /destinations/{slug}
 /visa
 /visa/{slug}
 /visa/apply
 /air-ticketing
 /gallery
 /blog
 /blog/{slug}
 /faq
 /contact
```

---

7. Legal Routes

```
/privacy-policy
/terms-and-conditions
/refund-cancellation-policy
/visa-disclaimer
/cookie-policy
```

---

8. Admin Routes

Use:

```
/admin
```

Modules:

```
/admin/dashboard

/admin/tours
/admin/tours/create
/admin/tours/{id}/edit

/admin/destinations

/admin/visa/countries
/admin/visa/types
/admin/visa/requirements
/admin/visa/applications

/admin/ticket-requests

/admin/inquiries

/admin/offers

/admin/gallery

/admin/testimonials

/admin/blog
/admin/blog/categories

/admin/faqs

/admin/pages
```

```
/admin/homepage

/admin/seo

/admin/media

/admin/users
/admin/roles

/admin/settings

/admin/activity-logs
```

Admin routes must be protected by authentication and authorization middleware.

---

9. API Architecture

API prefix:

```
/api/v1
```

Public API examples:

```
GET    /api/v1/tours
GET    /api/v1/tours/{slug}

GET    /api/v1/destinations
GET    /api/v1/destinations/{slug}

GET    /api/v1/visa/countries
GET    /api/v1/visa/{slug}

POST   /api/v1/inquiries
POST   /api/v1/ticket-requests
POST   /api/v1/visa-applications
POST   /api/v1/contact
```

Future:

```
/api/v2
```

must remain possible without breaking `/api/v1`.

---

10. Public Website Rendering Strategy

Default:

```
Laravel Blade
```

Use Vue only where interaction requires it.

Example:

```
Tour Listing
    ↓
Blade renders initial HTML
    ↓
Vue enhances filters
```

Instead of:

```
Empty HTML
    ↓
Vue loads everything
    ↓
API request
    ↓
Page appears
```

The first architecture is preferred for SEO and performance.

---

11. Vue Architecture

Recommended:

```
resources/js/
├── components/
│   ├── public/
│   └── admin/
│
├── pages/
├── composables/
├── stores/
├── services/
```

```
├── utils/
└── app.js
```

---

12. Vue Responsibilities

Vue components should handle:

```
Interactive filters
Search autocomplete
Multi-step visa form
Document upload
Dynamic profession requirements
Gallery lightbox
Inquiry forms
Admin tables
Admin forms
Admin dashboard
```

Vue should not duplicate business logic already implemented in Laravel.

---

13. State Management

Use local component state for simple interactions.

Use a centralized store only when state is shared across multiple components.

Potential shared states:

```
Admin authentication
Admin filters
Visa application draft
Global UI state
```

Avoid global state for static public content.

---

14. Database Architecture

Database:

```
MySQL
```

Use:

```
InnoDB
utf8mb4
utf8mb4_unicode_ci
```

All timestamps should use Laravel's standard timestamp handling.

---

15. Database Naming Convention

Use:

```
snake_case
```

Examples:

```
tour_categories
visa_countries
visa_applications
ticket_requests
created_at
updated_at
```

Primary key:

```
id
```

Foreign key:

```
destination_id
tour_category_id
visa_country_id
```

---

16. Core Database Entity Map

```
User
 |
```

```
  ├── Role
  │
  └── ActivityLog

Destination
  │
  ├── Tours
  │
  ├── VisaCountry
  │
  └── BlogPosts

TourCategory
  │
  └── Tours

Tour
  │
  ├── Itineraries
  ├── Inclusions
  ├── Exclusions
  ├── Images
  ├── FAQs
  └── Inquiries

VisaCountry
  │
  ├── VisaTypes
  │    └── Requirements
  │
  ├── ProfessionRequirements
  │
  └── Applications
       └── Documents

BlogCategory
  │
  └── BlogPosts

GalleryCategory
  │
  └── GalleryImages
```

---

17. Users Table

Fields:

```
id
name
email
phone
password
status
last_login_at
remember_token
created_at
updated_at
```

---

18. Roles Table

```
id
name
slug
description
created_at
updated_at
```

---

19. Permissions Table

```
id
name
slug
description
created_at
updated_at
```

---

20. Tour Categories

```
tour_categories

id
name
slug
description
image
status
sort_order
```

```
created_at
updated_at
```

Examples:

```
Family
Honeymoon
Group
Adventure
Luxury
Budget
Corporate
Customized
```

---

21. Destinations Table

```
destinations

id
name
slug
country
region
short_description
description
hero_image
thumbnail
status
featured
sort_order
created_at
updated_at
```

---

22. Tours Table

```
tours

id
destination_id
tour_category_id

title
slug
```

```
short_description
description

duration_days
duration_nights

price
currency

status
featured
popular

cover_image

published_at

created_at
updated_at
```

---

23. Tour Itineraries

```
tour_itineraries

id
tour_id

day_number
title
description

sort_order

created_at
updated_at
```

One tour:

```
Tour
 ├── Day 1
 ├── Day 2
 ├── Day 3
 └── Day N
```

---

24. Tour Inclusions

```
tour_inclusions

id
tour_id
title
description
sort_order
```

Examples:

```
Hotel Accommodation
Breakfast
Airport Transfer
Sightseeing
Guide
```

---

25. Tour Exclusions

```
tour_exclusions

id
tour_id
title
description
sort_order
```

---

26. Tour Images

Use a polymorphic media relationship where practical.

Concept:

```
media
 ├── Tour
 ├── Destination
 ├── Blog
 ├── Gallery
 └── Testimonial
```

27. Visa Countries

```
visa_countries

id
country_name
slug
flag
hero_image
short_description
description
status
featured
sort_order
created_at
updated_at
```

28. Visa Types

```
visa_types

id
visa_country_id
name
slug
description

processing_time
processing_time_unit

starting_fee
currency

service_fee
currency

status
sort_order

created_at
updated_at
```

Examples:

```
Tourist Visa
Business Visa
Student Visa
Medical Visa
Family Visa
E-Visa
```

---

29. Visa Requirements

```
visa_requirements

id
visa_type_id

title
description

required
sort_order

created_at
updated_at
```

---

30. Profession Types

```
profession_types

id
name
slug
status
sort_order
```

Examples:

```
Salaried
Business Owner
Student
Freelancer
Retired
```

```
Housewife
Other
```

---

31. Profession-Based Requirements

```
visa_profession_requirements

id
visa_type_id
profession_type_id

title
description
required

sort_order

created_at
updated_at
```

Relationship:

```
Visa Type
    ↓
Profession
    ↓
Recommended Documents
```

---

32. Visa Applications

```
visa_applications

id

reference_number

visa_country_id
visa_type_id

full_name
email
phone
date_of_birth
```

```
nationality
passport_number

travel_purpose
travel_date
stay_duration
travelers_count

profession_type_id

status

admin_notes

submitted_at
created_at
updated_at
```

---

33. Visa Application Status

Allowed states:

```
received
under_review
documents_required
processing
submitted
decision_received
completed
rejected
cancelled
```

Status transitions should be validated.

---

34. Visa Documents

```
visa_documents

id
visa_application_id

document_type
original_filename
stored_filename
```

```
mime_type
file_size
storage_path

status

uploaded_at
created_at
updated_at
```

Documents must use private storage.

---

35. Ticket Requests

```
ticket_requests

id

reference_number

trip_type

origin
destination

departure_date
return_date

adults
children
infants

cabin_class
preferred_airline

additional_requirements

full_name
email
phone

status

admin_notes

created_at
updated_at
```

36. Ticket Request Status

```
new
contacted
quote_prepared
waiting_for_customer
confirmed
closed
cancelled
```

37. Tour Inquiries

```
tour_inquiries

id

reference_number

tour_id

full_name
email
phone

travel_date
adults
children

message

status
assigned_to

admin_notes

created_at
updated_at
```

38. Contact Messages

```
contact_messages

id

name
email
phone
service

travel_date
message

status

created_at
updated_at
```

---

39. Offers

```
offers

id

title
slug
description

destination_id
tour_id

discount_type
discount_value

start_date
end_date

image

status
featured

created_at
updated_at
```

40. Testimonials

```
testimonials

id

customer_name
photo

destination
service

review
rating
source

featured
status

created_at
updated_at
```

41. Gallery Categories

```
gallery_categories

id
name
slug
description
status
sort_order
created_at
updated_at
```

42. Gallery Images

```
gallery_images

id
gallery_category_id
```

```
title
image_path
thumbnail_path
alt_text
caption

status
sort_order

created_at
updated_at
```

---

43. Blog Categories

```
blog_categories

id
name
slug
description
status
created_at
updated_at
```

---

44. Blog Posts

```
blog_posts

id
blog_category_id

author_id

title
slug

excerpt
content

featured_image

status
```

```
published_at
created_at
updated_at
```

---

45. FAQ

```
faqs

id

category
question
answer

related_type
related_id

status
sort_order

created_at
updated_at
```

Optional polymorphic relation:

```
Tour
Visa
General
```

---

46. Pages

For CMS-driven static pages:

```
pages

id

title
slug
content

status
```

```
created_at
updated_at
```

Examples:

```
About
Privacy Policy
Terms
Refund Policy
Visa Disclaimer
```

---

47. SEO Metadata

Recommended:

```
seo_metadata

id

seoable_type
seoable_id

meta_title
meta_description

focus_keyword

canonical_url

og_title
og_description
og_image

robots

schema_type

created_at
updated_at
```

Use polymorphic SEO relationships.

---

48. Media Table

```
media

id

mediable_type
mediable_id

disk

path
filename

mime_type
size

alt_text
caption

width
height

created_at
updated_at
```

---

49. Settings Table

```
settings

id
key
value
type
group
created_at
updated_at
```

Groups:

```
general
contact
social
seo
branding
```

```
homepage
email
notification
```

---

50. Activity Logs

```
activity_logs

id

user_id

action
subject_type
subject_id

description

ip_address
user_agent

created_at
updated_at
```

---

51. Reference Number System

Every major inquiry/application should receive a unique reference number.

Examples:

```
VISA-2026-000001
TOUR-2026-000001
TKT-2026-000001
INQ-2026-000001
```

Reference generation must be concurrency-safe.

---

52. Model Relationships

Destination

```
Destination
hasMany Tours
hasMany Offers
```

Tour

```
Tour
belongsTo Destination
belongsTo TourCategory
hasMany Itineraries
hasMany Inclusions
hasMany Exclusions
hasMany Inquiries
morphMany Media
morphOne SEO
```

VisaCountry

```
VisaCountry
hasMany VisaTypes
```

VisaType

```
VisaType
belongsTo VisaCountry
hasMany Requirements
hasMany ProfessionRequirements
hasMany Applications
```

---

53. Database Indexing

Required indexes:

```
destinations.slug
destinations.status

tours.slug
```

```
tours.destination_id
tours.tour_category_id
tours.status
tours.featured
tours.published_at

visa_countries.slug
visa_countries.status

visa_types.slug
visa_types.visa_country_id

visa_applications.reference_number
visa_applications.status
visa_applications.created_at

ticket_requests.reference_number
ticket_requests.status

tour_inquiries.reference_number
tour_inquiries.status

blog_posts.slug
blog_posts.status
blog_posts.published_at
```

Use unique indexes where appropriate.

---

54. Slug Strategy

All public entities require unique slugs.

Example:

```
Thailand Tour Package
↓
thailand-tour-package
```

Do not use database IDs in public URLs unless necessary.

---

55. Slug Collision Handling

If a slug exists:

```
thailand-tour-package-2
```

or another deterministic unique suffix.

When changing an existing published slug, create a redirect record.

---

56. Redirect Management

Optional table:

```
redirects

id
old_url
new_url
status_code
created_at
updated_at
```

Default:

```
301
```

Use for SEO-safe URL changes.

---

57. Service Layer

Controllers should remain thin.

Example:

```
public function store(StoreTourInquiryRequest $request)
{
    $inquiry = $this->tourInquiryService
        ->create($request->validated());

    return redirect()
        ->back()
        ->with('success', 'Inquiry submitted.');
}
```

Business logic belongs in:

```
TourInquiryService
```

---

58. Repository / Query Strategy

Repositories are optional where Laravel Eloquent is already sufficient.

Use repositories/query classes when:

- Queries become complex
- Multiple controllers share queries
- Reporting requires complex aggregation
- Search/filter logic becomes large

Do not create unnecessary abstraction layers for simple CRUD.

---

59. Form Requests

Every major form should have a dedicated Form Request.

Examples:

```
StoreTourInquiryRequest
StoreVisaApplicationRequest
StoreTicketRequest
StoreContactMessageRequest
StoreTourRequest
UpdateTourRequest
```

Validation belongs here.

---

60. Authorization

Use Laravel Policies/Gates.

Example:

```
TourPolicy
VisaPolicy
ApplicationPolicy
InquiryPolicy
BlogPolicy
```

```
GalleryPolicy
UserPolicy
```

Authorization must happen server-side.

---

61. Middleware

Recommended middleware:

```
auth
verified
admin
role
permission
throttle
```

Custom:

```
EnsureAdminAccess
TrackAdminActivity
```

---

62. Authentication

Admin authentication must include:

- Secure password hashing
- Session regeneration
- Login throttling
- Password reset
- Email verification where appropriate
- Session timeout
- Logout invalidation

Future-ready for 2FA.

---

63. Public Inquiry Security

Public forms should use:

```
CSRF
Rate Limiting
```

```
Honeypot
Validation
Duplicate Submission Protection
```

CAPTCHA should only be enabled where necessary.

---

64. File Upload Security

Allowed file types should be explicitly configured.

Example:

```
PDF
JPG
JPEG
PNG
WEBP
```

Maximum size must be configurable.

Validate:

```
MIME
Extension
File Size
Image Dimensions
```

Do not trust client-provided MIME types.

---

65. Private Document Storage

Visa documents:

```
storage/app/private/visa/
```

Never:

```
public/storage/visa/
```

unless files are intentionally public.

Downloads should be authorized and preferably use temporary/signed URLs.

---

66. Image Optimization Pipeline

When admin uploads an image:

```
Original
   ↓
Validation
   ↓
Optimization
   ↓
Generate WebP / AVIF
   ↓
Generate Thumbnail
   ↓
Store Metadata
```

Original files may be retained privately if needed.

---

67. Responsive Image Architecture

Generate or expose variants:

```
thumbnail
small
medium
large
original
```

Use:

```
srcset
sizes
```

where appropriate.

---

68. Caching Architecture

Cache:

```
Homepage
Navigation
Settings
Featured Tours
Popular Destinations
Visa Country Lists
FAQ
```

Cache should be invalidated when relevant content changes.

---

69. Laravel Cache

Use Laravel Cache abstraction.

Potential backend:

```
File
Database
Redis
```

Start with what the Hostinger plan reliably supports.

If Redis is available, use it for production caching and queues where appropriate.

---

70. Queue Architecture

Use Laravel Jobs for:

```
Email Notifications
Image Processing
Heavy Reports
Bulk Notifications
Future API Synchronization
```

Do not block a customer-facing HTTP request with expensive operations.

---

71. Notification Architecture

Create notification classes:

```
NewInquiryNotification
NewVisaApplicationNotification
NewTicketRequestNotification
VisaStatusUpdatedNotification
```

Support:

```
Mail
Database
Future WhatsApp/SMS
```

---

72. Email Architecture

Use reusable templates.

```
resources/views/emails/
├── layouts/
├── inquiries/
├── visa/
├── tickets/
└── system/
```

---

73. Search Architecture

MVP:

Use MySQL indexed search.

Search across:

```
Tours
Destinations
Visa Countries
Blog Posts
```

Future:

```
Meilisearch
Typesense
Algolia
```

Do not introduce an external search engine unless the content scale requires it.

---

74. Tour Filtering

Filtering should support:

```
destination
country
category
duration
price
status
featured
```

Queries must be optimized.

Use query scopes:

```
scopePublished()
scopeFeatured()
scopePopular()
scopeByDestination()
```

---

75. Visa Filtering

Support:

```
country
visa_type
status
featured
```

---

76. Pagination

Use pagination for all potentially large datasets.

Examples:

```
Tours: 12/page
Blog: 9/page
Gallery: 24/page
Applications: 20/page
Inquiries: 20/page
```

Exact values can be adjusted after testing.

---

77. Admin Search

Admin should support:

```
Global Search
Tour Search
Visa Application Search
Inquiry Search
Ticket Request Search
Customer Search
Blog Search
```

Reference numbers should be directly searchable.

---

78. Admin Filters

Applications:

```
Status
Visa Country
Visa Type
Date Range
Assigned Staff
```

Inquiries:

```
Service
Status
Date
Assigned Staff
```

---

79. Customer Data Model

MVP does not require full customer accounts.

Customer information can be stored within:

```
visa_applications
tour_inquiries
ticket_requests
contact_messages
```

Future customer account system can normalize this into:

```
customers
```

---

80. Future Customer Table

Potential Phase 2:

```
customers

id
name
email
phone
password
status
created_at
updated_at
```

Then applications/inquiries can belong to customers.

---

81. Booking Architecture

MVP:

```
Inquiry-based
```

Not:

```
Full instant booking engine
```

This keeps MVP simpler and safer.

Future:

```
Inquiry
↓
Quote
↓
Booking
↓
Payment
↓
Confirmation
```

---

82. Quote System — Future

Potential tables:

```
quotes
quote_items
quote_versions
```

Workflow:

```
Inquiry
↓
Staff prepares quote
↓
Customer receives quote
↓
Customer accepts
↓
Booking
```

---

83. Payment Architecture — Future

Payment should NOT be implemented directly into the MVP architecture unless required.

Future providers can be integrated through a payment abstraction:

```
PaymentService
    ↓
PaymentGatewayInterface
    ├── Gateway A
    ├── Gateway B
    └── Gateway C
```

Never couple core booking logic directly to one payment provider.

---

84. SEO Architecture

Every public indexable entity should support:

```
SEO title
Meta description
Canonical
Open Graph
Robots
Schema
```

Laravel should render these server-side.

---

85. SEO Rendering

Each page layout:

```
<head>

<title>{{ seo.title }}</title>

<meta name="description">

<link rel="canonical">

<meta property="og:title">
<meta property="og:description">
<meta property="og:image">

</head>
```

---

86. Structured Data

Create reusable JSON-LD generators.

Potential classes:

```
OrganizationSchema
TravelAgencySchema
BreadcrumbSchema
TourSchema
ArticleSchema
FAQSchema
```

Only output applicable schema.

---

87. Sitemap Architecture

Dynamic sitemap generator:

```
/sitemap.xml
```

May include:

```
/sitemap-pages.xml
/sitemap-tours.xml
/sitemap-destinations.xml
/sitemap-visa.xml
/sitemap-blog.xml
```

A sitemap index can be used if the number of URLs grows substantially.

---

88. Robots Architecture

Robots rules should protect:

```
/admin
/private
/internal
```

Do not accidentally block:

```
/tours
/visa
/destinations
/blog
```

---

89. Breadcrumb Architecture

Generate breadcrumbs dynamically.

Example:

```
Home
→ Tour Packages
→ Thailand
→ Thailand Escape
```

Breadcrumb data should also be available for schema generation.

---

90. Content Versioning — Recommended

For important visa information, consider keeping:

```
updated_at
updated_by
```

and optionally version history.

This allows staff to identify who changed sensitive travel information.

---

91. Visa Information Freshness

Visa processing information can change.

Admin should display:

```
Last Updated:
16 September 2026
```

where appropriate.

Potential field:

```
information_updated_at
```

---

92. Disclaimer Architecture

Visa pages should support reusable disclaimer blocks.

Example:

```
Visa information may change without notice.
Approval is determined by the relevant authority.
```

The disclaimer should be editable from admin settings.

---

93. Settings Architecture

Global settings:

```
Brand Name
Logo
Favicon
Phone
WhatsApp
Email
Address
Business Hours
Social Links
Google Maps
Footer Text
Default SEO
Analytics IDs
```

---

94. Environment Variables

Example:

```
APP_NAME=
APP_ENV=
```

```
APP_KEY=
APP_DEBUG=false
APP_URL=

DB_CONNECTION=mysql
DB_HOST=
DB_PORT=
DB_DATABASE=
DB_USERNAME=
DB_PASSWORD=

MAIL_MAILER=
MAIL_HOST=
MAIL_PORT=
MAIL_USERNAME=
MAIL_PASSWORD=
MAIL_ENCRYPTION=
MAIL_FROM_ADDRESS=
MAIL_FROM_NAME=

ANALYTICS_ID=
META_PIXEL_ID=
```

Never commit `.env`.

---

95. Environment Separation

Use:

```
.env.local
.env.staging
.env.production
```

or equivalent deployment environment management.

Production secrets must never be stored in Git.

---

96. Logging

Laravel logs should capture:

- Exceptions
- Failed jobs
- Critical application errors
- Security-related events

Do not log sensitive visa document contents.

Avoid logging:

```
Passport numbers
Bank details
Passwords
Private documents
```

---

97. Error Handling

Production:

```
APP_DEBUG=false
```

Use custom:

```
404
403
419
422
429
500
503
```

pages/responses.

---

98. API Error Format

Use consistent JSON:

```
{
  "success": false,
  "message": "Unable to submit the application.",
  "errors": {}
}
```

Do not expose internal exceptions.

---

99. API Success Format

```
{
  "success": true,
  "message": "Application submitted successfully.",
  "data": {
    "reference_number": "VISA-2026-000001"
  }
}
```

---

100. API Rate Limiting

Public API:

```
throttle
```

Sensitive endpoints:

```
POST /visa-applications
POST /inquiries
POST /ticket-requests
```

should have stricter limits.

---

101. CSRF

All browser-based state-changing forms must use Laravel CSRF protection.

API authentication strategy must be selected separately for API clients.

---

102. XSS Prevention

Content editors may use rich text.

Sanitize HTML before rendering where necessary.

Never blindly output user-submitted HTML.

---

103. SQL Injection Prevention

Use:

```
Eloquent
Query Builder
Parameterized Queries
```

Never construct raw SQL from untrusted user input.

---

104. Authorization Principle

Never rely on hidden frontend buttons for security.

Example:

Hiding:

```
Delete button
```

is NOT authorization.

Backend must independently check:

```
$user->can('delete', $tour)
```

---

105. Admin Activity Tracking

Track important actions:

```
Create
Update
Delete
Publish
Unpublish
Status Change
Document Access
User Permission Change
Settings Change
```

---

106. Data Retention

Define retention policies for:

```
Visa Documents
Contact Messages
Old Inquiries
Activity Logs
```

Sensitive documents should not be retained indefinitely without business/legal need.

---

107. Backup Strategy

Production backups should include:

```
Database
Uploaded Media
Configuration backup where appropriate
```

Recommended:

```
Daily database backup
Regular media backup
Periodic restore test
```

Do not assume hosting backups alone are sufficient.

---

108. Deployment Architecture

```
Developer
   ↓
GitHub
   ↓
Production Build
   ↓
Hostinger
   ↓
Laravel
   ↓
MySQL
```

Production deployment should run:

```
composer install --no-dev --optimize-autoloader
npm install
npm run build

php artisan migrate --force

php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Commands should be adapted to the actual Hostinger environment.

---

109. Deployment Safety

Before migration:

```
Database backup
```

Before major release:

```
Staging test
```

After deployment:

```
Health check
Homepage test
Tour test
Visa test
Form test
Admin test
```

---

110. Git Strategy

Recommended:

```
main
develop
feature/*
```

```
fix/*
hotfix/*
```

Production deploys from:

```
main
```

---

111. Commit Convention

Examples:

```
feat: add visa application wizard
feat: add tour package filters
fix: resolve mobile navigation issue
perf: optimize destination queries
seo: add tour schema
security: protect visa documents
refactor: extract inquiry service
```

---

112. Testing Strategy

Testing layers:

```
Unit Tests
Feature Tests
Integration Tests
Browser / E2E Tests
Manual Responsive Tests
Performance Tests
Security Tests
```

---

113. Unit Tests

Test:

```
Reference number generation
Price calculations
Status transitions
Slug generation
```

```
Visa requirement selection
Filtering logic
```

---

114. Feature Tests

Test:

```
Tour creation
Tour publishing
Visa application submission
Ticket inquiry
Contact form
Admin authorization
Document upload
SEO metadata
```

---

115. E2E Tests

Critical user flows:

```
Visitor
→ Homepage
→ Tour
→ Tour Detail
→ Inquiry
→ Success

Visitor
→ Visa
→ Country
→ Apply
→ Upload Documents
→ Submit

Admin
→ Login
→ View Application
→ Change Status
→ Save
```

---

116. Performance Testing

Test:

```
Homepage
Tour Listing
Tour Detail
Visa Listing
Visa Detail
Blog
Admin Dashboard
```

Measure:

```
LCP
INP
CLS
TTFB
Page Weight
JavaScript Size
Image Weight
```

---

117. Query Optimization

Avoid:

```
N+1
Repeated queries
Unbounded queries
SELECT *
```

Use:

```
select()
with()
withCount()
paginate()
```

where appropriate.

---

118. SEO + Performance Rule

Do not sacrifice SEO for JavaScript convenience.

Preferred:

```
Server-render content first
Enhance with Vue second
```

---

119. Third-Party Script Strategy

Third-party scripts should be limited.

Potential:

```
GA4
Meta Pixel
Google Maps
WhatsApp
```

Load them responsibly.

Do not load unnecessary tracking libraries.

---

120. Google Maps

Avoid embedding a huge map iframe on every page.

Use:

```
Contact page only
```

or lazy-load map interaction.

---

121. WhatsApp

Use a simple click-to-chat URL.

Do not load an unnecessary JavaScript SDK just for a WhatsApp button.

122. Analytics

Analytics events:

```
view_tour
view_visa
submit_tour_inquiry
start_visa_application
submit_visa_application
submit_ticket_request
submit_contact_form
click_whatsapp
click_phone
click_email
```

123. SEO-Friendly Blog Architecture

Blog URLs:

```
/blog/{slug}
```

Related content:

```
Blog
 ↓
Destination
 ↓
Tour
 ↓
Visa
```

Use contextual internal links.

124. Content Relationships

A tour can be associated with:

```
Destination
Blog Posts
Visa
```

```
FAQs
Gallery
Offers
```

This creates a strong internal content graph.

---

125. Example Content Graph

```
THAILAND
│
├── Thailand Tour Package
│
├── Thailand Visa
│
├── Thailand Travel Guide
│
├── Thailand Gallery
│
└── Thailand FAQ
```

This structure is useful for both UX and SEO.

---

126. Admin Homepage Architecture

Homepage content should be dynamic.

Admin controls:

```
Hero
Services
Featured Tours
Destinations
Visa Countries
Offers
Why Us
How It Works
Testimonials
Blog
FAQ
Final CTA
```

Each section supports:

```
enabled
sort_order
title
subtitle
content
selected_items
```

---

127. Feature Flags

For future development, optional features may use configuration/feature flags.

Examples:

```
online_booking
payments
customer_accounts
flight_api
hotel_api
multilingual
```

Do not build a complex feature-flag platform for MVP.

---

128. Scalability Strategy

MVP:

```
Laravel Monolith
+
MySQL
+
Blade
+
Selective Vue
```

Future scale:

```
Laravel
+
Redis
+
Queue Workers
+
```

```
CDN
+
Search Engine
+
External APIs
```

Do NOT begin with microservices.

---

129. Why Modular Monolith

Advantages:

- Easier Hostinger deployment
- Easier development
- Lower operational complexity
- Better maintainability
- Faster MVP
- Easier debugging
- Easy future extraction of services

---

130. Future API Integrations

Architecture should allow:

```
Flight API
Hotel API
Payment Gateway
WhatsApp API
SMS Gateway
Email Provider
Maps
Travel Insurance
CRM
```

Integrations should live under:

```
app/Integrations/
```

Example:

```
app/Integrations/
├── Flights/
├── Hotels/
```

```
├── Payments/
├── WhatsApp/
└── Sms/
```

---

131. Integration Interface Pattern

Example:

```
FlightProviderInterface

    ↓

AmadeusProvider
SabreProvider
TravelportProvider
```

This prevents vendor lock-in.

---

132. Admin Dashboard API

Admin Vue may consume Laravel API endpoints or use server-rendered routes with progressive enhancement.

Preferred for complex interactive admin tables:

```
Vue
 ↓
Laravel API
 ↓
Service
 ↓
Database
```

---

133. Public Forms

For normal public forms, Laravel server-side submission is preferred.

Example:

```
Blade Form
↓
POST
↓
Laravel Controller
↓
Form Request
↓
Service
↓
Database
↓
Notification
↓
Redirect
```

Vue enhancement is optional.

---

134. Visa Application Wizard

For the complex visa application:

```
Vue
 ↓
Local Draft State
 ↓
Step Validation
 ↓
Final Submit
 ↓
Laravel
 ↓
Database
 ↓
Private Documents
```

Do not persist incomplete sensitive data unnecessarily.

---

135. Draft Application Strategy

If draft saving is required later:

```
visa_application_drafts
```

But MVP should preferably keep unfinished forms client-side until submission unless business requirements require server-side drafts.

---

136. Document Upload Strategy

Preferred:

```
Select file
↓
Client validation
↓
Upload
↓
Temporary/private storage
↓
Final submission
↓
Attach document to application
```

Temporary uploads must expire automatically.

---

137. Sensitive Data Principle

Minimize storage of sensitive information.

Only collect information required for the service.

Do not collect:

```
Passwords
Unnecessary financial information
Unnecessary personal data
```

---

138. Privacy Architecture

Privacy policy should explain:

- What information is collected
- Why it is collected
- How it is stored
- Who may access it
- How long it is retained

• How customers can request appropriate data handling

---

139. Accessibility Architecture

Use:

```
Semantic HTML
ARIA only when necessary
Keyboard navigation
Accessible forms
Visible focus
Proper labels
Alt text
```

---

140. Localization Architecture

Future-ready:

```
resources/lang/en
resources/lang/bn
```

Content database can later support translation tables.

MVP may launch with one language.

---

141. Multi-Currency Architecture

Use:

```
price
currency
```

instead of hardcoding ৳ inside database values.

Primary:

```
BDT
```

Future:

```
USD
EUR
GBP
AED
```

---

142. Timezone

Application timezone should be configured explicitly.

For Bangladesh:

```
Asia/Dhaka
```

All database timestamps should be stored consistently, with display converted to the configured application/user timezone where needed.

---

143. File Naming

Avoid user-controlled raw filenames for storage.

Use:

```
UUID / hash-based filenames
```

Keep original filename as metadata only.

---

144. Admin Media Permissions

Permissions:

```
media.view
media.upload
media.edit
media.delete
```

Visa document access should be separately permission-controlled.

---

145. API Authentication

Public APIs:

```
No authentication
+
Rate limiting
```

Admin APIs:

```
Authenticated
+
Authorization
```

Future mobile app:

```
Laravel Sanctum
```

or an appropriate token strategy.

---

146. Cache Invalidation

When a tour changes:

```
Clear Tour Cache
Clear Destination Related Cache
Clear Homepage Featured Cache
```

When visa data changes:

```
Clear Visa Cache
Clear Related Country Cache
```

---

147. Publishing Workflow

Content:

```
Draft
↓
```

```
Review
↓
Publish
```

Optional:

```
Scheduled
```

Only published content appears publicly.

---

148. Soft Deletes

Use soft deletes where historical references matter.

Potential:

```
Tours
Blog Posts
Visa Types
Applications
Inquiries
```

Do not use soft deletes everywhere automatically.

---

149. Data Integrity

Foreign keys should use appropriate constraints.

Example:

```
tour.destination_id
→ destinations.id
```

Choose delete behavior carefully.

For important historical records:

```
RESTRICT
```

may be preferable to cascading deletion.

---

150. Transaction Strategy

Use database transactions for multi-step writes.

Example:

```
Visa Application
+
Documents
+
Reference Number
+
Initial Status
```

must be created safely.

---

151. Idempotency

For public submissions, prevent duplicate submissions caused by:

- Double-click
- Browser retry
- Network retry

Use:

```
Submission token
Idempotency key
Duplicate detection
```

where appropriate.

---

152. Admin Dashboard Metrics

Queries should aggregate:

```
Total Tours
Active Tours
Tour Inquiries
Visa Applications
Ticket Requests
```

```
Contact Messages
Popular Destinations
```

Use optimized aggregate queries.

Do not load thousands of records just to calculate a count.

---

153. Reporting Architecture

Future reporting can use:

```
reports
analytics_events
```

MVP can rely on external analytics plus dashboard aggregates.

---

154. SEO-Friendly Pagination

For listing pages:

```
/tours?page=2
```

is acceptable.

Do not create indexable filter combinations unnecessarily.

Canonicalization must prevent duplicate crawl paths.

---

155. Search URL Strategy

Search results should generally not become indexable SEO pages unless deliberately designed as landing pages.

Example:

```
/search?q=thailand
```

should usually be:

```
noindex
```

---

156. Admin Security Headers

Configure appropriate headers:

```
Content-Security-Policy
X-Content-Type-Options
Referrer-Policy
Permissions-Policy
Frame protection
```

Exact CSP must account for legitimate external services.

---

157. HTTPS

Production must use:

```
HTTPS
```

HTTP should redirect to HTTPS.

Cookies should use:

```
Secure
HttpOnly
SameSite
```

where appropriate.

---

158. Production Configuration

Production:

```
APP_ENV=production
APP_DEBUG=false
```

Cache:

```
config
routes
views
```

Optimize Composer autoloading.

---

159. Health Check

Provide a lightweight internal health endpoint.

Example:

```
/health
```

It should verify basic application availability without exposing sensitive information.

---

160. Maintenance Mode

Laravel maintenance mode should be used during risky deployments.

Do not expose debug information.

---

161. Architecture Decision Record

Decision 01

Laravel Monolith

Decision: Use Laravel modular monolith.

Reason: MVP simplicity, Hostinger compatibility, maintainability and scalability.

Decision 02

Blade + Selective Vue

Decision: Do not build the public website as a full SPA.

Reason: SEO + performance + lower JavaScript overhead.

Decision 03

Inquiry-Based MVP

Decision: Use inquiry workflows instead of full instant booking.

Reason: Lower complexity and aligns with travel agency consultation workflow.

Decision 04

Private Visa Documents

Decision: Store visa documents privately.

Reason: Sensitive customer information requires controlled access.

Decision 05

Modular Integration Layer

Decision: External APIs must use service/provider abstractions.

Reason: Future vendor replacement and scalability.

---

162. Recommended Project Structure

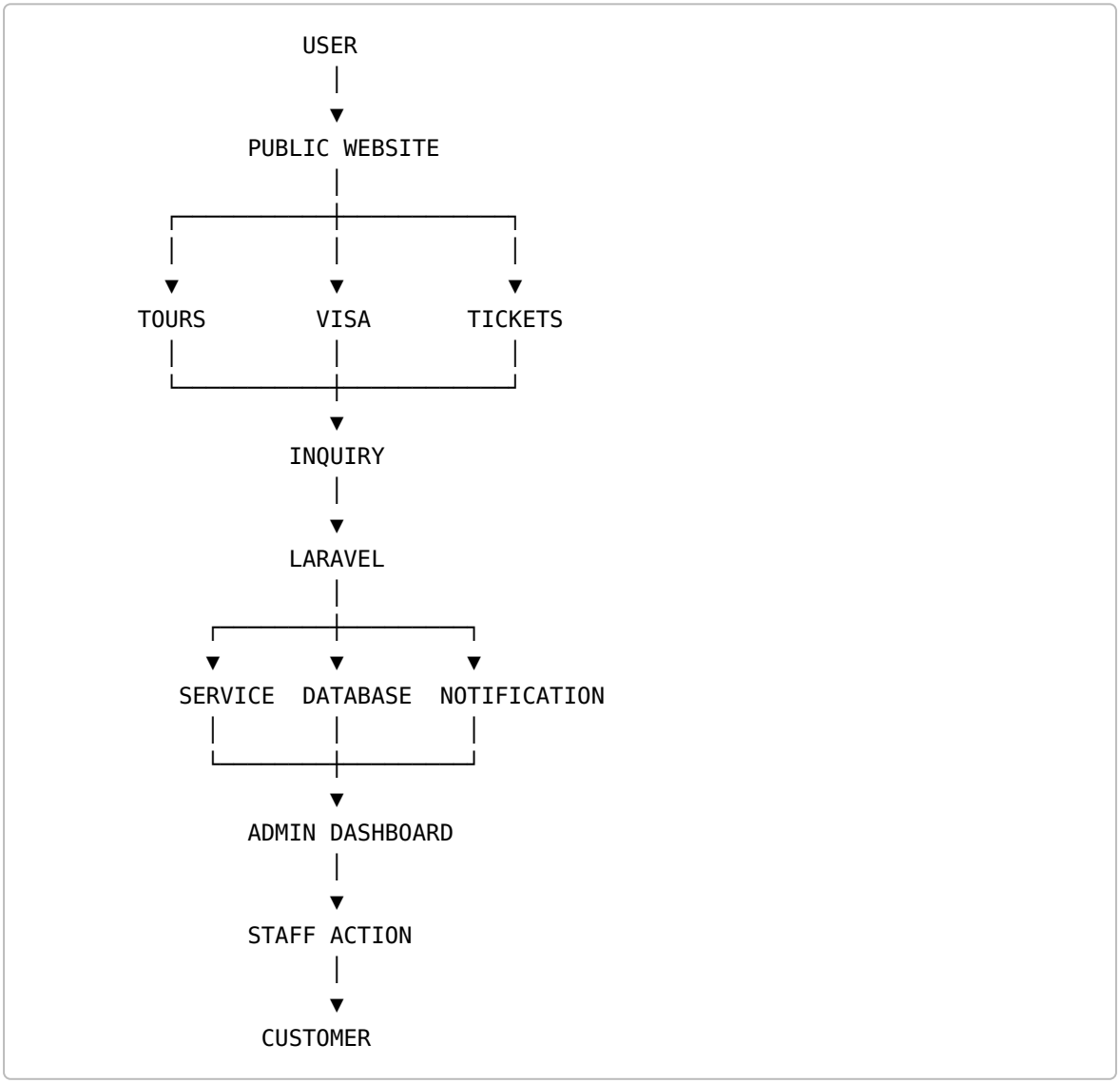
```
travel-agency/

├── app/
│   ├── Console/
│   ├── Exceptions/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Public/
│   │   │   ├── Admin/
│   │   │   └── Api/
│   │   ├── Middleware/
│   │   └── Requests/
│   │
│   ├── Models/
│   ├── Services/
│   ├── Repositories/
│   ├── Policies/
│   ├── Notifications/
│   ├── Jobs/
```

```
│   ├── Events/
│   ├── Listeners/
│   ├── Integrations/
│   └── Support/
│
├── database/
│   ├── migrations/
│   ├── seeders/
│   └── factories/
│
├── resources/
│   ├── views/
│   │   ├── layouts/
│   │   ├── components/
│   │   ├── public/
│   │   ├── admin/
│   │   └── emails/
│   │
│   ├── js/
│   │   ├── components/
│   │   │   ├── public/
│   │   │   └── admin/
│   │   ├── pages/
│   │   ├── composables/
│   │   ├── stores/
│   │   ├── services/
│   │   └── utils/
│   │
│   └── css/
│
├── routes/
│   ├── web.php
│   ├── admin.php
│   ├── api.php
│   └── console.php
│
├── storage/
│   ├── app/
│   │   ├── private/
│   │   │   └── visa/
│   │   └── public/
│   └── logs/
│
├── tests/
│   ├── Unit/
│   ├── Feature/
│   └── Browser/
│
└── public/
```

163. Final Architecture Flow

The final application should operate like this:



---

164. Final Architecture Principles

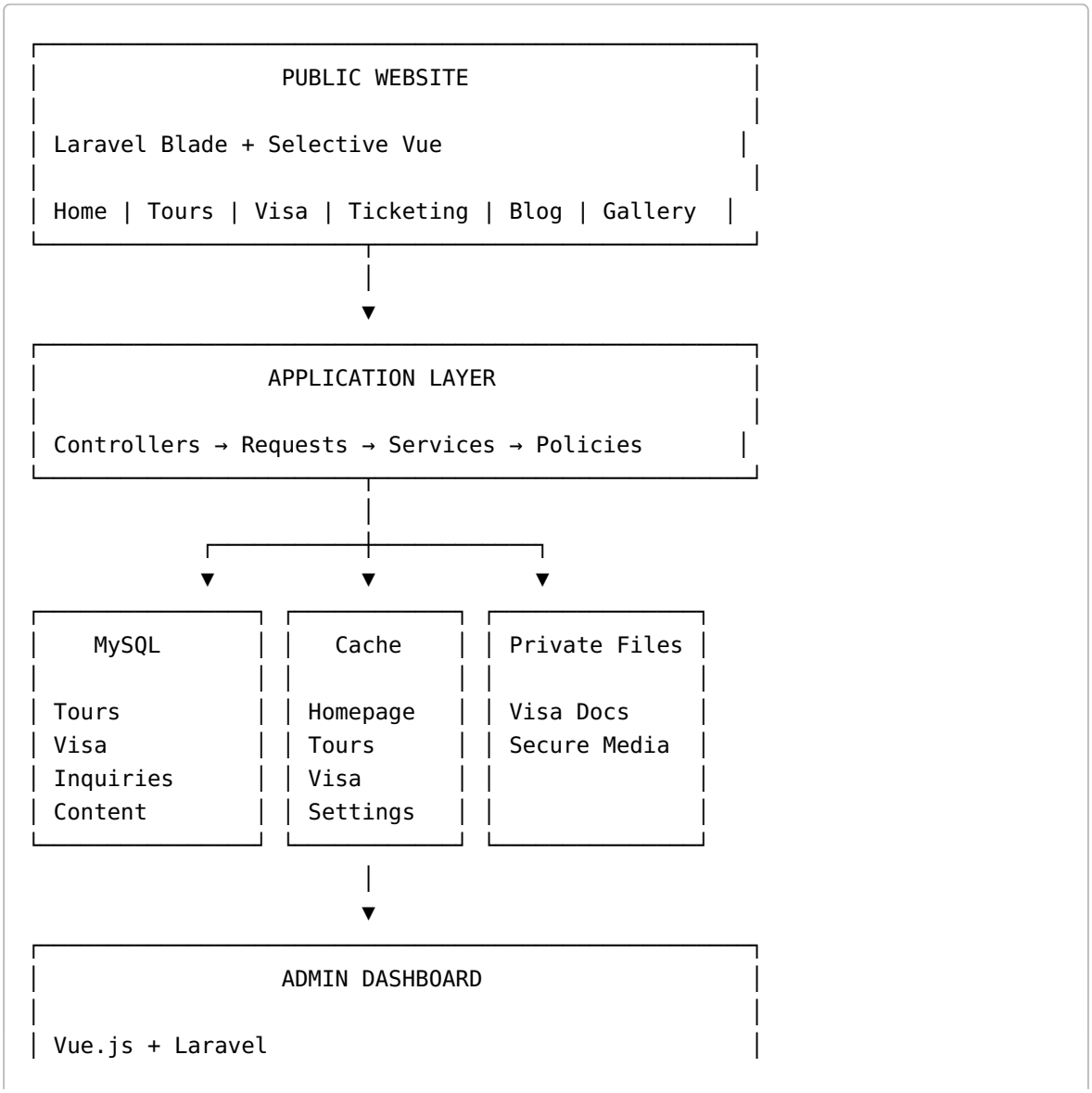
The implementation must follow these rules:

1. Laravel remains the core application framework.
2. MySQL remains the primary relational database.
3. Blade is the default public rendering layer.
4. Vue is used selectively for interactivity.
5. Admin dashboard can use Vue extensively.
6. Business logic belongs in services, not views.
7. Controllers remain thin.

8. Sensitive documents remain private.
9. All public content must be SEO-friendly.
10. All important entities use clean slugs.
11. Database queries must be optimized.
12. Images must be optimized.
13. Third-party scripts must be minimized.
14. Public forms require server-side validation.
15. Admin actions require server-side authorization.
16. External integrations must be abstracted.
17. MVP should remain an inquiry-based system.
18. The architecture should be modular without becoming unnecessarily complex.
19. The system must be mobile-first and responsive.
20. The application must remain ready for future booking, payment, flight API, hotel API and mobile-app integrations.

---

165. Final System Blueprint



```
|                                                          |
| CMS | Tours | Visa | Applications | Inquiries      |
| SEO | Gallery | Blog | Users | Settings            |
└──────────────────────────────────────────────────────────┘
```

END OF ARCHITECTURE.md